

**Genetic Algorithms for Stock Trading:
Time-Weighted and Adaptive operation**

WONG Yan Kit

Under the Guidance of:

Prof. Benjamin W. Wah, Research Professor, Department of CSE, CUHK

Abstract

This project explores two innovative approaches to enhancing genetic algorithms for stock trading. The first approach aims to enhance NEAT algorithms to improve the generation of stock trading signals. It proposed Time-Weighted Adaptive NEAT based on the assumption that more recent stock data are more closely correlated with unseen data. This approach seeks to better understand non-stationary stock price data trends for improved generalization. By making NEAT's operations more adaptive to the learning data, we addressed the issue of random probing.

The second approach explores an innovative approach of leveraging genetic algorithms to augment the neural network topology in a fast-growing way, named MEANT (Modularly Enhanced Augmenting Neural Topology). Extending the traditional methods, such as NEAT, which augment the neural network topology by adding single node and edge, MEANT advances by integrating entire modules. This approach aims to address the limitations of NEAT's slower, incremental growth, and the inability to match the complexity of modern, large-scale models. MEANT investigates the feasibility of autonomously determining optimal network structure, particularly in complex domains like stock trading.

Together, these approaches signify promising advancements in algorithmic trading strategies, with the Time-Weighted Adaptive NEAT improving risk-adjusted returns and generalization capabilities and MEANT focusing on accelerating the evolutionary process and enhancing adaptability.

1 Introduction

This project aims to optimize the topology of neural network responsible for generating the buy/sell signals through NEAT (NeuroEvolution of Augmenting Topologies), building a neural network capable of determining threshold values of technique indicators for the signal generation. In traditional approaches, it often involves manually designing the neural network architecture, while for stock trading tasks, there are no 'true labels' or 'true values' available to construct a closed form for the optimization of the neural network. Hence, the objective space is not differentiable. Common approaches, including gradient descent, cannot be utilized. NEAT as an evolutionary algorithm, it introduces dynamically evolving network structures based on performance without requiring close form optimization.

The second part of this project introduces a method that extend the idea of augmenting topology in NEAT by employing a modular approach to the network topology named Modularly Enhanced Augmenting Neural Topology (MEANT). Traditionally, neural network design relies on human expertise and are determined by trial and errors and trained in a predefined architecture and input parameters. Some other methods, like NEAT, attempt to adjust the complexity of a neural network incrementally through techniques of augmenting topologies. However, such techniques fall short when scaling up to the complexity required for modern application. It potentially accelerates the evolutionary process and enhances the adaptability of the neural network, making it more compatible for complex tasks like predictive analysis in stock trading.

Together, these approaches seek to leverage the strength of each method to develop a more profitable and adaptive trading system, showcasing the potential of deep learning techniques to enhance the decision-making in quantitative finance markets and address the challenges in algorithmic trading.

2 Literature Review

The paper “Evolving Neural Networks through Augmenting Topologies” introduces an optimization algorithm evolving neural network topologies along with weights. Genetic algorithm is a search and optimization algorithm using the idea of natural selection. There is a population that has a set of genomes. The optimization operations including selection, crossover, and mutation operations. Fitness function is used to measure how good each genome is and only genomes perform in a good way will be kept. Crossover will try to combine two genomes to produce a new offspring and mutation will add or delete node and connection. NEAT is the genetic algorithm that designed for evolving artificial neural networks.

The paper "A Deep Neural Network Based Stock Trading System Based on Evolutionary Optimized Technical Analysis" proposes a stock trading system that uses genetic algorithms to find the optimal RSI threshold for signal generation. It utilizes 10 years of price data for training the neural network and subsequent 5 years of price data for testing. This paper could serve as the fundamental framework for the trading system in this project.

Furthermore, in the paper "Improving the Performance of NEAT-Related Algorithms via Complexity Reduction in the Search Space," it is demonstrated that by reducing the crossover rate, NEAT can retain more information to learn more effectively. This suggests a direction for enhancing the operations of NEAT. To address non-stationary stock market data, the paper "Dynamic Ensemble of Ensembles in Nonstationary Environments" proposes a weighted majority voting approach to combine classifiers as an ensemble of all interim ensembles dynamically from consecutive batches of non-stationary data. It presents an approach to learning non-stationary data for machine learning.

The concepts presented in these papers will serve as the foundation for the trading framework and addressing non-stationary data to enhance generalization.

3 Problem statement

This project focuses on addressing three main challenges of stock trading with NEAT:

Non-Stationary in Stock Market

The statistical properties such as mean and variance change over time and it is difficult to do generalization, which is the ability of a trained model to perform how well on unseen data . It is challenging to build an accurate machine learning model since the pattern learned in training phase may not be persist in the future. The project seeks to find a way to deal with non-stationary in generalization, enabling neural networks to accurately predict outcomes on the unseen data.

Random Probing in Evolutionary Algorithm

The mutation, crossover operations are determined by a fixed probability, resulting in a stochastic exploration of the search space, and finding the optimal topology could be costly due to the reliance on random exploration. The project aims to develop an approach to enhances the adaptability of models to effectively learn from training data, guiding the search process and to be more efficient and targeted exploration techniques.

Slow Network Growing Speed in NEAT

In the original NEAT paper, perceptrons in the neural networks are added one by one in each evolution. It could help achieve a simple neural network structure but fail to grow into a complex model capable of handling large-scale, modern applications. Consequently, it limits the network's ability to effectively address intricate problems in domains like stock trading.

4 Time-Weighted Adaptive NEAT

Dataset & Data Preprocessing

The closing prices of DOWJ30 stocks on a 1-Day timeframe from January 1, 2013, to December 31, 2022, spanning 10 years, will be used. The dataset will be retrieved using the Tiingo API, with each stock in the DOWJ30 having its individual file. Hence, 30 files will be downloaded, totaling approximately 7MB. There are no null or missing values.

The correlation between the stock 'AAPL' and others is plotted to select 6 of them for simplicity. Specifically, 'AAPL', the second-highest correlated stock, median correlated stocks, the second lowest, and the lowest correlated stocks, to explore a diverse portfolio. In this way, NEAT's performance under various conditions could be assessed.

The selected stocks are {'AAPL', 'MSFT', 'GS', 'JPM', 'DD', 'GE'}

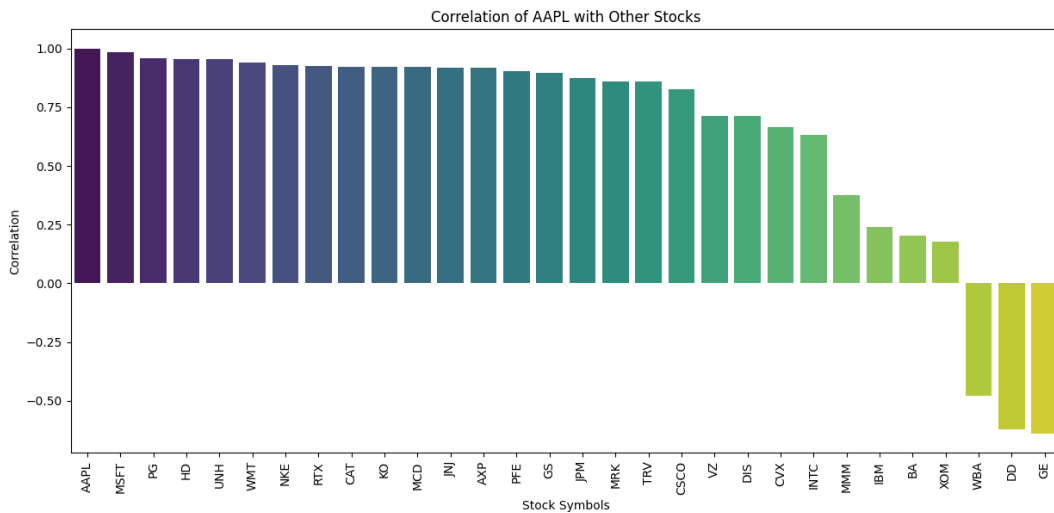


Fig 1: The correlation between AAPL stock and others

Moving Training and Testing Windows

The system will iterate through different stock data, and moving training and testing windows will be applied during the 10-year period. The daily return (the change in stock price) will be used as input for the neural network within the windows since it is normalized and the stock prices might exhibit significant variations over the specified time range.

Trading Rule

We would have two independent model for long strategy and short strategy respectively. The number of input neurons are the same with training windows size, and scaled daily return would be the input data. Different genomes represent different neural networks, and each neural network would have two outputs:

Output[0]:	RSI threshold for opening position
Output[1]:	RSI threshold for closing position

These outputs would be used to the parameters of technical indicators and thresholds for opening position or closing position signal.

In this project, two technical indicators are used for the trading rules:

Exponential Moving Average (EMA)

EMA is a type of moving average that gives more weight to recent data points, making it more responsive to changes in the underlying data.

$$EMA_t = \alpha \times \text{Current Value}_t + (1 - \alpha) \times EMA_t$$

where α is the smoothing factor.

Relative Strength Index (RSI)

RSI is a momentum oscillator that measures the speed and change of price movements, identifying overbought or oversold conditions in a financial asset.

$$RSI = 100 - \frac{100}{1 + RS}$$

where RS (Relative Strength) is the average of n days' up values divided by the average of n days down values:

Long Strategy	Short Strategy
<u>Open Long Position</u> 1. Daily return > 3-day EMA of Daily Return AND 2. RSI < output[0] <u>Close Long Position</u> 1. RSI > output[1]	<u>Open Short Position</u> 1. Daily return < 3-day EMA of Daily Return AND 2. RSI > output[1] <u>Close Short Position</u> 1. RSI < output[0]

Table 1: The trading rules of long and short strategies

Fitness Value

The fitness value is measured by the Sortino ratio, considering the risk-adjusted return during the testing period. The formula of Sortino ratio:

$$fitness\ value = Sortino\ Ratio = \frac{r_p - r_f}{\sigma_d}$$

where r_p is the return of the stock in the testing period

r_f is the risk-free rate (0 in this project)

σ_d is the standard deviation of downside returns

Assumption on the Correlation

The paper "Dynamic Ensemble of Ensembles in Nonstationary Environments" introduces weighted majority voting could potentially deal with non-stationary learning. In this project, it is assumed that more recent stock trend data have a higher influence on the unseen stock trend data, hence the genomes would be time-weighted to capture a better learning to generalize the unseen stock trend.

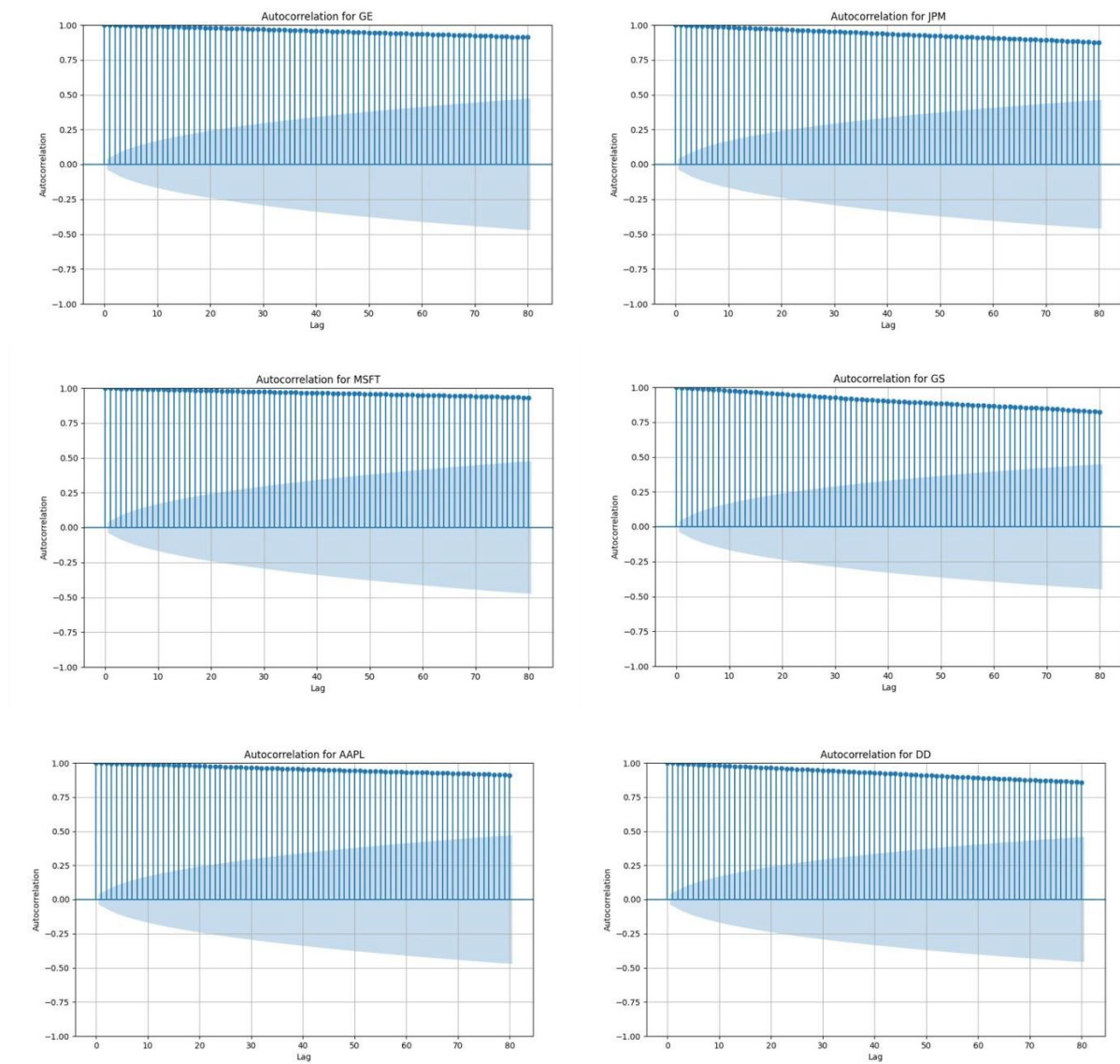
Autocorrelation function will be used to measure similarity between observations in a time series data at different time points, where 1 indicates a perfect positive correlation and -1 indicates a perfect negative correlation. Hence autocorrelation function will capture the overall correlation pattern.

The formula of the autocorrelation function:

$$\hat{\rho}_k = \frac{\sum_{t=k+1}^T (r_t - \bar{r})(r_{t-k} - \bar{r})}{\sum_{t=1}^T (r_t - \bar{r})^2}$$

This project is trying to learn the strength of the trend in non-stationary environment; therefore, it is not necessary to difference the data.

Here are the graphs of the autocorrelation of each stock:



The above autocorrelation plot, the ACF decays slowly. It means there is significant correlation between observations at nearby lags. Hence, data have a strong time dependence, future values of the series are correlated or heavily affected by past values. To conclude, more recent data is higher correlated to future unseen data, making the more recent neural networks have larger impact, a better prediction to future is expected to be made.

Non-stationarity and Time-weighting

Under the assumption of “More recent data correlates highly with the unseen stock data”. Time-weighting is applied to the genomes in this project. The probabilities of doing the operations (mutation/crossover) change with each generation. In each generation, the probability of each operation is independent to the others, and time-weighting is applied to these probabilities. The more recent probabilities have relatively larger weighting, while the less recent probabilities have relatively smaller weighting. This idea also applies to the prediction outcome, the prediction outcomes of the more recent generation of genomes have a relative larger weighting, the less recent generation of genomes have a relative smaller weighting. Therefore, the more recent data have a large impact to the learning, enabling better generalization to unseen data under the assumption.

The actual probability of this generation is the sum of normalized time-weighting of all probabilities.

The formula of the final probability of doing the operation of genome k:

$$P_k = \frac{w_1 p_1 + w_2 p_2 + \dots + w_k p_k}{w_1 + w_2 + \dots + w_k}$$

where p_k is the probability of genome k and w_k is the weighting of time k

The formula of the probability of the prediction outcome of genome k:

$$Y_k = \frac{w_1 y_1 + w_2 y_2 + \dots + w_k y_k}{w_1 + w_2 + \dots + w_k}$$

where y_k is the probability of genome k and w_k is the weighting of time k

Feedback channel between fitness and operation

The original method of determining the probability of doing operation is by random. It is lacking guarantees about the quality or efficiency of the solution found by algorithm. To improve the random probing, a feedback channel between fitness value and the probability of doing operation is built. A new attribute “mutation rate” is added for each genome. After each generation iteration, we would calculate the mutation rate of each genome:

Genome 1	Genome 2	...	Genome n
key	key		key
connections	connections		connections
nodes	nodes		nodes
fitness value	fitness value		fitness value
mutation rate	mutation rate		mutation rate

Table 4: The properties of genomes

$$mutation\ rate_i = 1 - mutation\ rate_i * \frac{fitness\ value\ of\ genome_i}{\sum_{i=0}^n fitness\ value\ of\ genome_i}$$

Hence, the mutation rate of a genome will be inversely proportional to its fitness value relative to the total fitness value among all genomes. In other words, if a genome has a relatively higher fitness value, it tends not to mutate to maintain its performance. Conversely, if a genome has a relatively lower fitness value, it tends to mutate to explore new possible solutions.

Delving deeper into the gene level, there are four types of mutations: adding a node, deleting a node, adding a connection, and deleting a connection. Additionally, it is aimed to establish a feedback channel between the fitness value and the gene. After a mutation occurs in the previous generation, there is a change in fitness value. Therefore, we can assess the effects of the gene on the fitness value.

Gene1	Gene 2	Gene n
key	key	key
fitness value before mutation	fitness value before mutation	fitness value before mutation
fitness value after mutation	fitness value after mutation	fitness value after mutation
fitness value change	fitness value change	fitness value change

Table 5 The properties of genes

We would calculate the total fitness change among all genes:

$$\Delta \text{fitness value of gene}_{i,k} = \text{fitness value of gene}_{i,k} - \text{fitness value of gene}_{i,k-1}$$

$$\text{Total fintess change}_k = \frac{\sum_{i=0}^n \text{fitness value of gene}_{i,k-1}}{\sum_{i=0}^n \text{fitness value of gene}_{i,k}}$$

where k is the number of generations

Then we would calculate the probabilities of different mutation operation:

$$P(\text{adding nodes})_{i,k} = \text{mutation rate}_{i,k} * (1 - (\frac{\Delta \text{fitness value of gene}_{i,k}}{\text{total fintess change}_k}))$$

$$P(\text{adding connection})_{i,k} = \text{mutation rate}_{i,k} * (1 - (\frac{\Delta \text{fitness value of gene}_{i,k}}{\text{total fintess change}_k}))$$

$$P(\text{deleting nodes})_{i,k} = \text{mutation rate}_{i,k} * (1 - (\frac{\Delta \text{fitness value of gene}_{i,k}}{\text{total fintess change}_k}))$$

$$P(\text{deleting connection})_{i,k} = \text{mutation rate}_{i,k} * (1 - (\frac{\Delta \text{fitness value of gene}_{i,k}}{\text{Total fintess change}_k}))$$

Therefore, the probabilities of different mutation operations of genomes will be inversely proportional to the change in their fitness value relative to the total change in fitness value among all genomes. In other words, after the mutation operation, if a genome experiences a relatively higher change in fitness value, it tends not to undergo mutation to maintain its

performance. Conversely, if a genome experiences a relatively lower change in fitness value, it tends to mutate to explore new possible solutions.

Genes that are present in both parents are inherited randomly with an equal chance. The focus is on disjoint and excess genes. Parent 1 is the genome with the higher fitness value, and Parent 2 is the genome with the lower fitness value. The additional genes that remain after the matching process are called disjoint genes and excess genes, respectively.

The probability of disjoint gene and excess gene to inherit:

$$P(\text{disjoint to inherit}) = \frac{\text{fitness value of Parent 1}}{\text{fitness value of Parent 1} + \text{fitness value of Parent 2}}$$

$$P(\text{excess to inherit}) = \frac{\text{fitness value of Parent 2}}{\text{fitness value of Parent 1} + \text{fitness value of Parent 2}}$$

Hence, the probability of inheriting disjoint and excess genes would be proportional to the fitness value of the parent. If the fitness value of the parent is relatively higher, we would tend to retain the extra genes to maintain its performance.

Results

The original NEAT is used to be compared with Time-weighted adaptive NEAT under the same date range and random seed.

$$\text{Annualized Return} = \left(\frac{\text{Ending Portfolio Value}}{\text{Beginning Portfolio Value}} \right)^{\frac{1}{\text{number of year}}} - 1$$

$$\text{Annualized Downside Volatility} = \sqrt[n]{\frac{1}{\text{number of year}} \sum_{t=1}^{\text{number of year}} [\min(r_t - \text{threshold value}, 0)]^2}$$

The results below evaluate the profitability and risk in unseen data testing period.

Original NEAT performance:

Long Strategy

Stock	Annualized Return	Average Number of Trade per year	Win Rate	Annualized Downside Volatility
AAPL	7.163%	14.7	69.388%	67.681%
MSFT	15.101%	15.1	78.808%	49.777%
GS	3.969%	7.8	67.949%	99.334%
JPM	3.354%	13.3	70.677%	74.832%
DD	0.832%	17.1	69.591%	134.100%
GE	0.292%	10.8	64.815%	143.683%
Average	5.12%	13.1	70.20%	94.900%

Table 6: The result of Long Strategy of Time-weighted adaptive NEAT

Short Strategy

Stock	Annualized Return	Average Number of Trade per year	Win Rate	Annualized Downside Volatility
AAPL	-9.443%	10.4	56.731%	53.278%
MSFT	-6.831%	17.8	55.056%	47.058%
GS	-3.615%	9.4	56.383%	56.298%
JPM	-7.197%	7.5	44.000%	37.756%
DD	2.689%	11.1	62.162%	68.913%
GE	0.870%	10.7	71.028%	74.136%
Average	-3.92%	11.2	57.560%	56.24%

Table 7: The result of Short Strategy of Time-weighted adaptive NEAT

Time-weighted adaptive NEAT performance:

Long Strategy

Stock	Annualized Return	Average Number of Trade per year	Win Rate	Annualized Downside Volatility
AAPL	12.238%	15.1	69.536%	67.681%
MSFT	21.510%	19.9	79.397%	49.777%
GS	8.737%	10.6	71.698%	99.334%
JPM	8.252%	11.4	71.930%	74.832%
DD	4.052%	13.0	72.308%	134.100%
GE	2.00%	9.7	63.918%	143.683%
Average	9.46%	13.3	71.46%	94.90%

Table 8: The result of Long Strategy of Original NEAT

Short Strategy

Stock	Annualized Return	Average Number of Trade per year	Win Rate	Annualized Downside Volatility
AAPL	-7.911%	10.2	50%	37.661%
MSFT	-1.505%	15.0	61.333%	38.566%
GS	-1.103%	13.0	60.769%	40.542%
JPM	-2.334%	10.1	55.446%	29.806%
DD	5.748%	10.3	66.990%	73.964%
GE	1.908%	10.0	72.000%	88.648%
Average	-0.865%	11.4	61.09%	51.53%

Table 9: The result of Short Strategy of Original NEAT

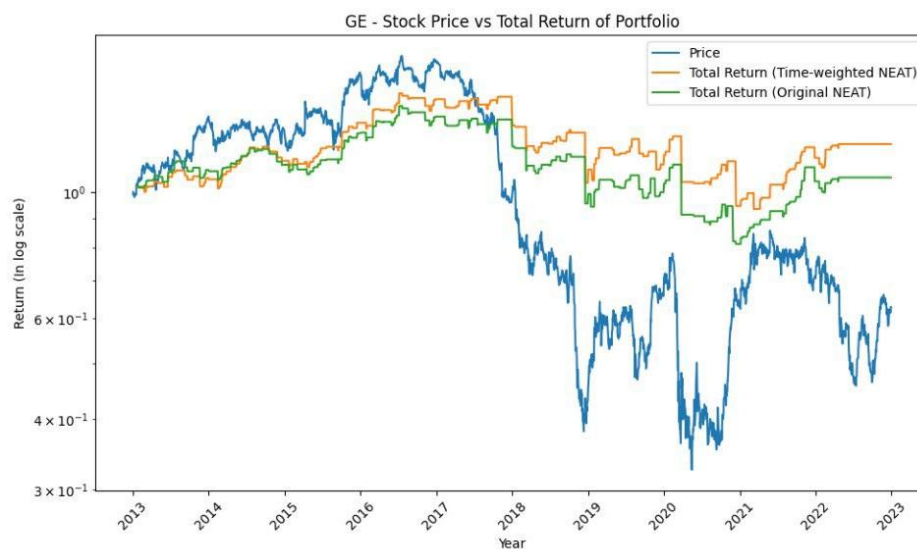
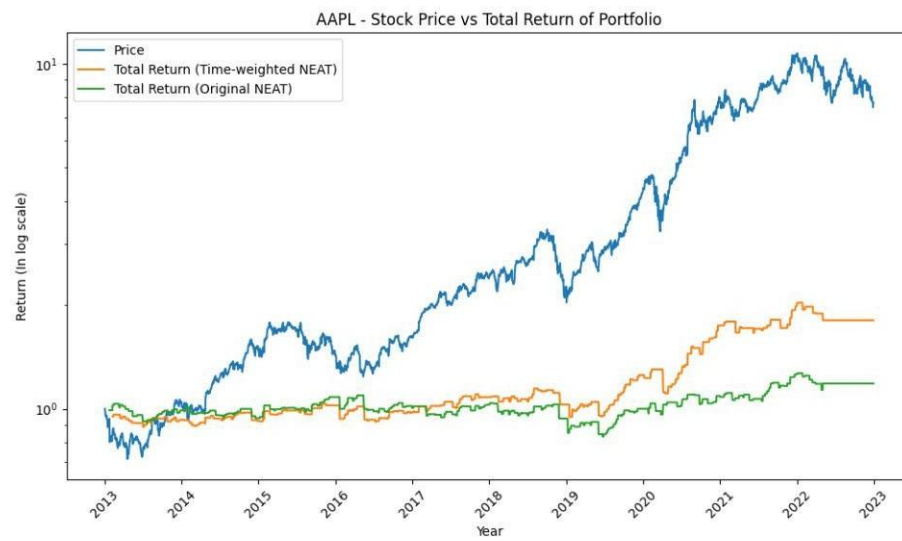
Difference of their profitability performance:

	Long Strategy			
	Annualized Return	Annualized Downside Volatility	Average Number of Trade per year	Win Rate
Time-weighted adaptive NEAT	9.46%	71.46%	13.3	71.46%
Original NEAT	5.12%	94.90%	13.1	70.20%
Difference	4.34%	-23.44%	0.2	1.26%

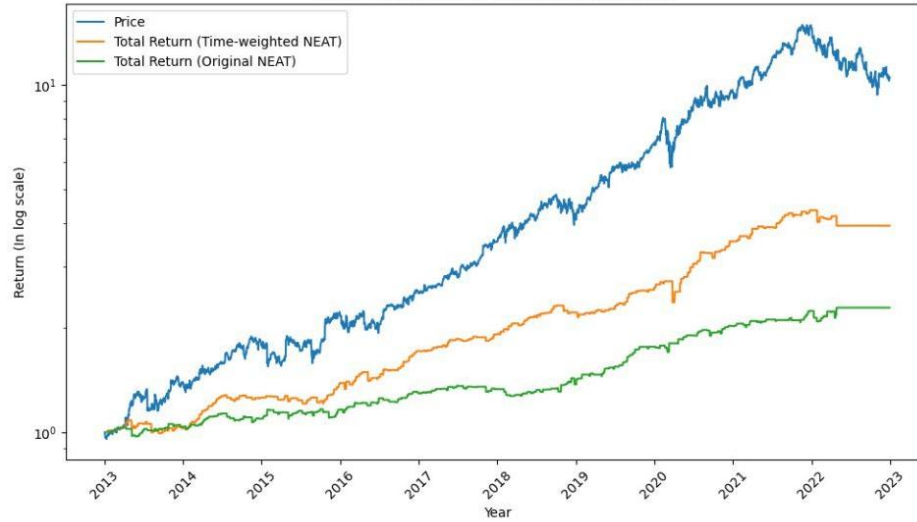
	Short Strategy			
	Annualized Return	Annualized Downside Volatility	Average Number of Trade per year	Win Rate
Time-weighted adaptive NEAT	-0.865%	51.53%	11.4	61.09%
Original NEAT	-3.92%	56.24%	11.2	57.560%
Difference	3.06%	-4.71%	0.2	3.53%

Table 10: The compared risk-adjust profitability result of Time-weighted adaptive NEAT and Original NEAT

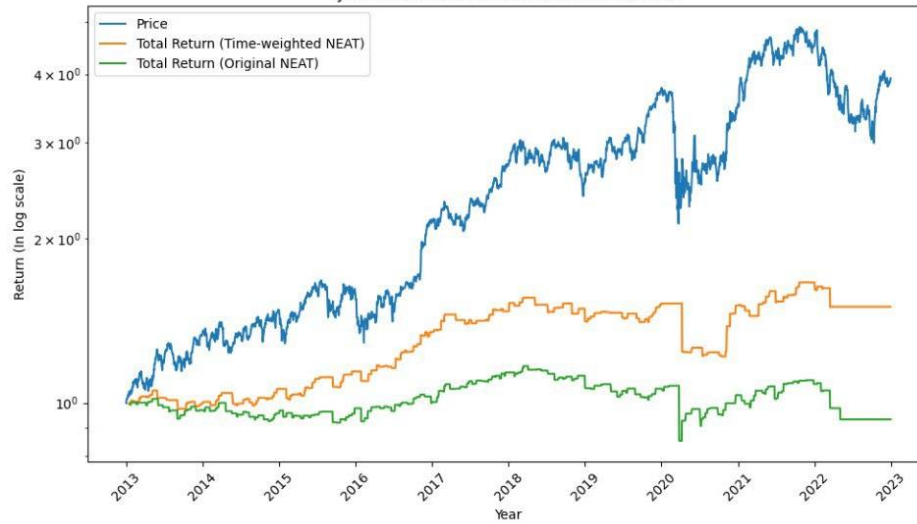
These are the plot of the total return of our approach and original approach with comparison of stock price movement.



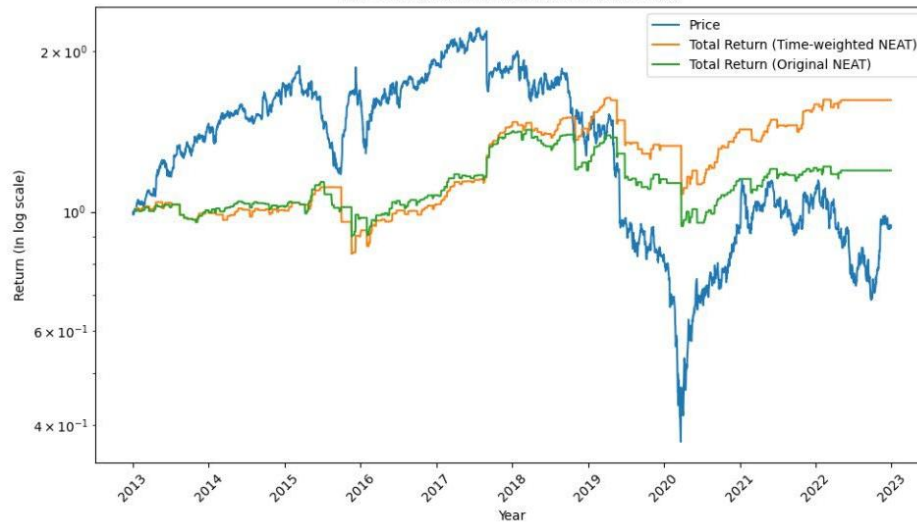
MSFT - Stock Price vs Total Return of Portfolio



JPM - Stock Price vs Total Return of Portfolio

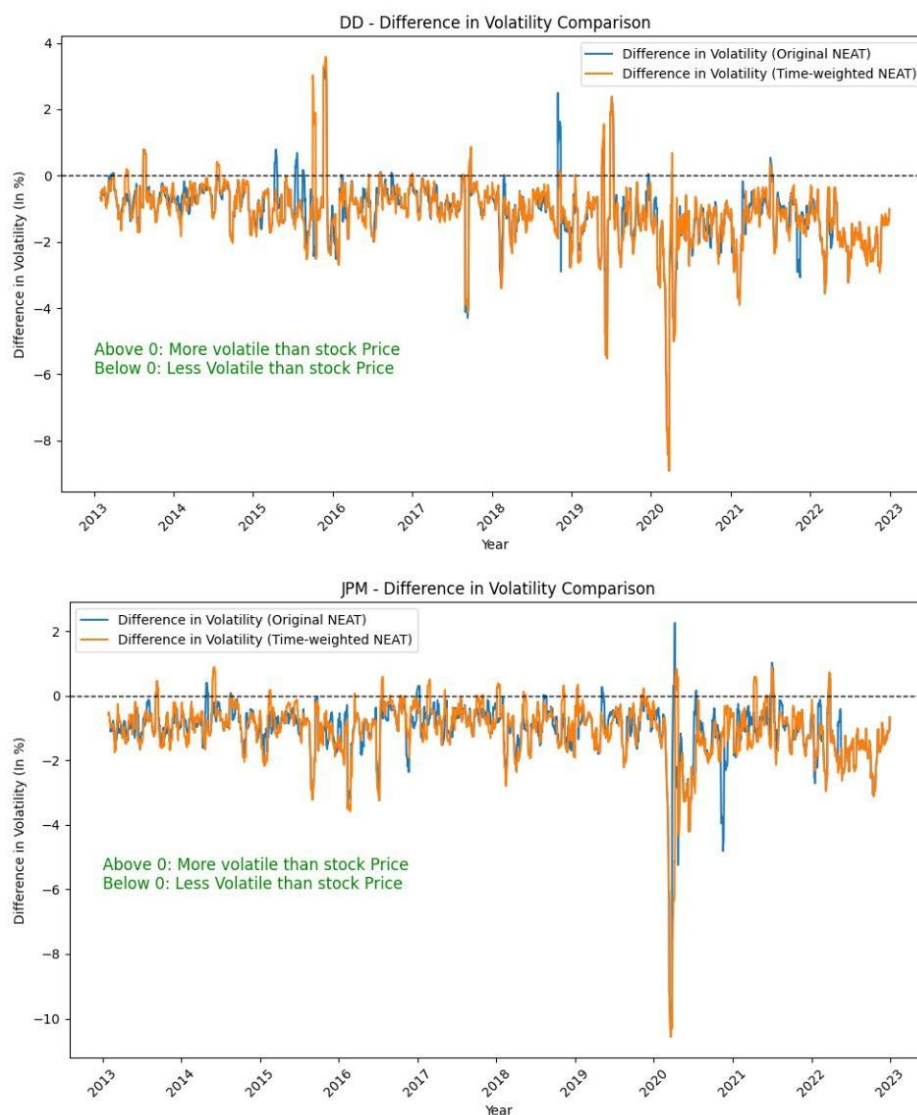


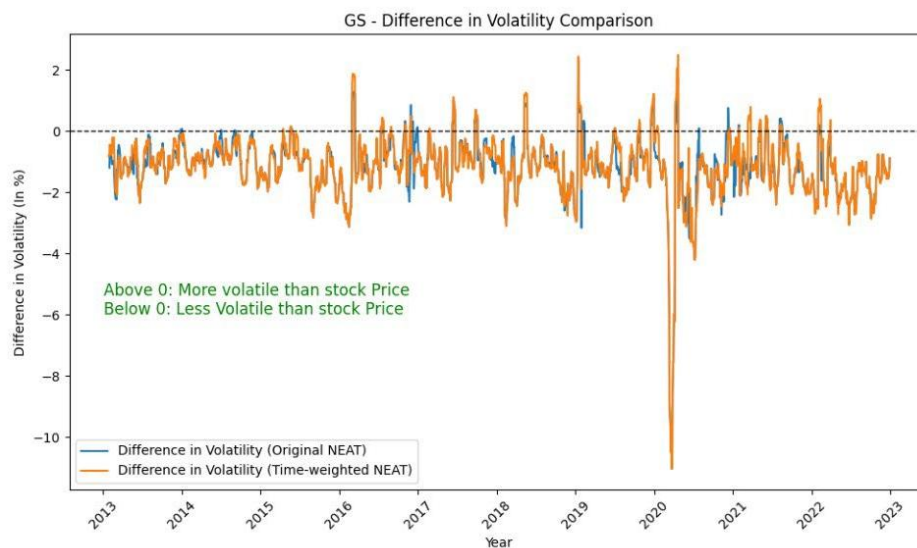
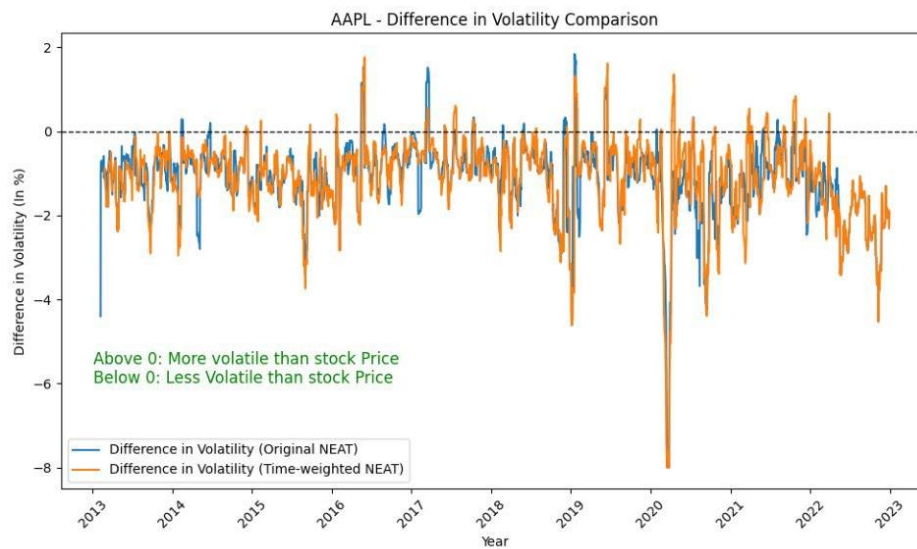
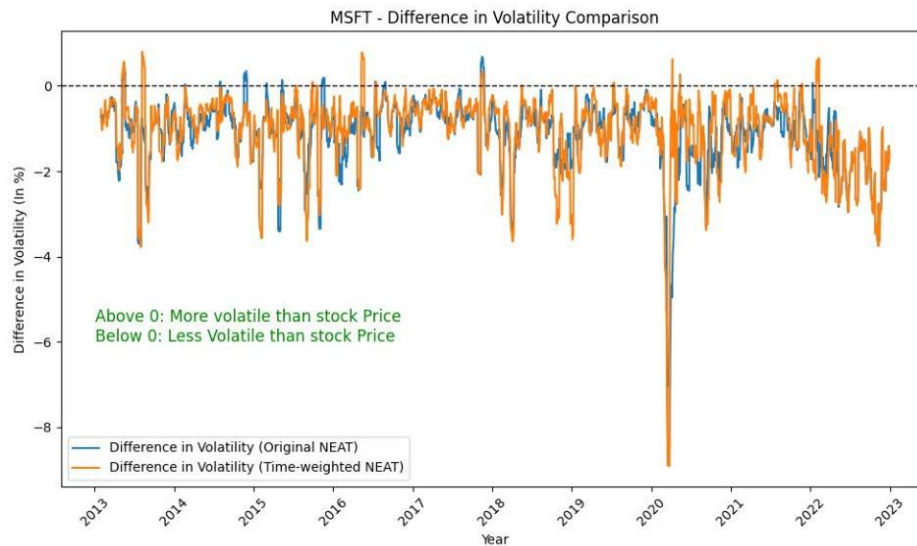
DD - Stock Price vs Total Return of Portfolio

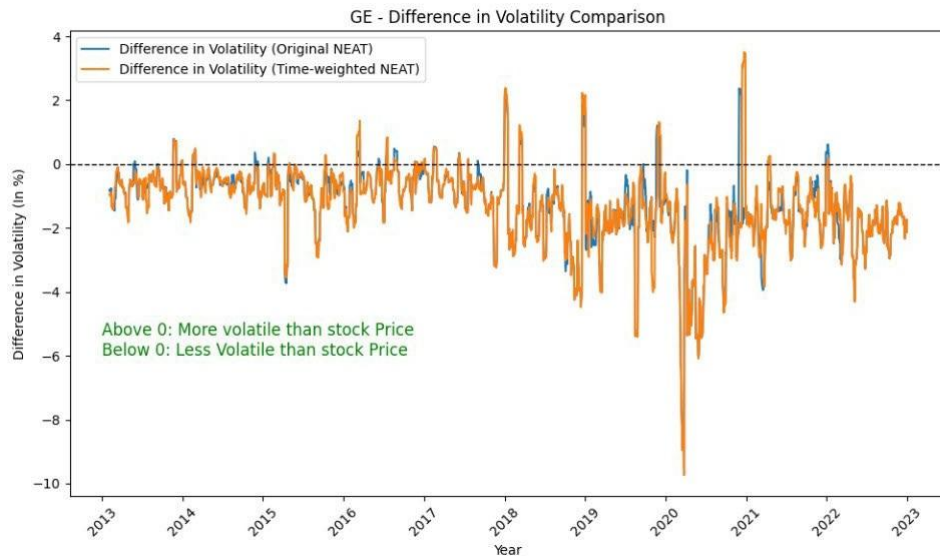


From the above plots, the total return of our approach is higher than the original approach in the majority of time. When there is a sharp drop of stock price, our approach still remains a lower volatility.

The below are the plot of the difference of comparing the volatility between stock price and Time-weighted adaptive NEAT approach, volatility between stock price and the original NEAT approach. If it is above the zero line, it means it is more volatile than stock price. If it is below the zero line, it means it is less volatile than stock price.







From the above plots, we can conclude that Time-weighted adaptive NEAT has a lower volatility than the stock price in major of time period, hence the risk of our approach is lower.

From the results above, it can be concluded that Time-weighted Adaptive NEAT achieves higher profits and lower risks in both long and short strategies. The risk-adjusted profitability of Time-weighted Adaptive NEAT surpasses that of the original NEAT. With more trades executed by Time-weighted Adaptive NEAT, it is believed that the time-weighting mechanism and adaptive operations enable the model to be more sensitive and better capture market trends.

The results below evaluate the generalization to unseen data. The average return per trade during the training period is based on the neural network with the best fitness value. This value is calculated by dividing the total return during the training period by the number of trades within that period and then taking the average across all training windows.

Original NEAT performance:

	Long Strategy – Average return per trade		Short Strategy – Average return per trade	
Stock	Training period	Testing period	Training period	Testing period
AAPL	0.82%	0.551%	-0.695%	-0.871%
MSFT	1.16%	0.955%	-0.347%	-0.344%
GS	0.95%	0.609%	-0.242%	-0.331%
JPM	0.74%	0.335%	-0.885%	-0.951%
DD	0.91%	0.213%	0.361%	0.337%
GE	0.65%	0.363%	0.246%	0.156%
Average	0.962%	0.504%	-0.08%	-0.33%
Difference	0.457%		0.26%	

Table 11: The result of Average return per trade of Time-weighted adaptive NEAT

Time-weighted adaptive NEAT performance:

	Long Strategy – Average return per trade		Short Strategy – Average return per trade	
Stock	Training period	Testing period	Training period	Testing period
AAPL	1.151%	0.870%	-0.672%	-0.767%
MSFT	1.222%	1.02%	0.058%	-0.0737%
GS	1.133%	0.958%	0.264%	-0.0563%
JPM	1.011%	0.799%	-0.012%	-0.213%
DD	0.962%	0.555%	0.806%	0.656%
GE	0.762%	0.560%	0.539%	0.296%
Average	1.040%	0.79%	0.164%	-0.026%
Difference	0.247%		0.190%	

Table 12: The result of Average return per trade of Original NEAT

Difference between their generalization ability:

	Long Strategy	Short Strategy
Time-weighted adaptive NEAT	0.247%	0.190%
Original NEAT	0.457%	0.260%
Difference	-0.21%	-0.07%

Table 13: The compared generalization ability result of Time-weighted adaptive NEAT and Original NEAT

Given the smaller difference in average return per trade between the training and testing phases, it can be concluded that Time-weighted Adaptive NEAT demonstrates superior generalization capabilities compared to the Original NEAT.

From the statistics provided, it can be concluded that Time-weighted Adaptive NEAT achieves higher returns, a greater number of trades, a higher win rate, and lower downside volatility. Additionally, this

method captures a better trend in more recent data, leading to improved risk-adjusted returns and an increase in trade volume. Using the same random seed and training period, Time-weighted Adaptive NEAT demonstrates higher returns, lower downside volatility, and a smaller discrepancy in the average return per trade between the training and testing periods. Therefore, it can be inferred that this approach offers better generalization to unseen data. Furthermore, the enhanced capture of recent trends indicates superior risk-adjusted returns and more frequent trading.

While the annualized return of Time-weighted Adaptive NEAT may not show considerable profitability, it highlights the potential benefits of utilizing time-weighting mechanisms and adaptive strategies to enhance trading signal generation.

6 Conclusion

This project has demonstrated that time-weighting and adaptive operations can address issues of non-stationarity and random probing in the stock trading market based on the tested hypothesis that "more recent data trends are more correlated with unseen data trends." Time-weighting prioritizes recent trend data and models to generate outcomes, while the feedback loop between the fitness value and the probability of crossover and mutation guides the genomes to explore the optimal solution more effectively and efficiently. This approach enables the model to generate higher risk-adjusted returns and to be more sensitive in capturing trends while its generalization ability surpasses that of the original NEAT.

Overall, the enhancements introduced by Time-Weighted Adaptive NEAT signify promising advancements in algorithmic trading strategies. Time-Weighted Adaptive NEAT improves risk-adjusted returns and generalization capabilities. This method showcase the potential of using advanced genetic algorithms to develop more robust and adaptive trading systems, enhancing decision-making in quantitative finance markets.

Reference

- [1] Sezer, Ö. B., Özbayoğlu, A. M., & Doğdu, E. (2017). A deep Neural-Network based stock trading system based on evolutionary optimized technical analysis parameters. *Procedia Computer Science*, 114, 473–480.
<https://doi.org/10.1016/j.procs.2017.09.031>
- [2] Mohabeer, H., & Soyjaudah, K. M. S. (2013). Improving the performance of NEAT related algorithm via complexity reduction in search space. In *Advances in intelligent systems and computing* (pp. 1–7).
https://doi.org/10.1007/978-3-319-00551-5_1
- [3] Yin, X., Huang, K., & Hao, H. (2013). Dynamic ensemble of ensembles in nonstationary environments. In *Lecture notes in computer science* (pp. 76–83).
https://doi.org/10.1007/978-3-642-42042-9_10
- [4] Shi, Y., Li, B., Long, W., & Dai, W. (2021). Method for improving the performance of technical analysis indicators by neural network models. *Computational Economics*, 59(3), 1027–1068. <https://doi.org/10.1007/s10614-021-10116-7>
- [5] Guresen, E., Kayakutlu, G., & Daim, T. U. (2011). Using artificial neural network models in stock market index prediction. *Expert Systems with Applications*, 38(8), 10389–10397.
- [6] Liao, Z., & Wang, J. (2010). Forecasting model of global stock index by stochastic time effective neural network. *Expert Systems with Applications*, 37(1), 834–841.
- [7] Bicheng Wang, Xinyi Zhang (2021). *Deep Learning Applying on Stock Trading*.
- [8] Khamesian, S., & Malek, H. (2021). Hybrid Self-Attention NEAT: A novel evolutionary approach to improve the NEAT algorithm. *arXiv (Cornell University)*.
<https://doi.org/10.48550/arxiv.2112.03670>
- [9] Wang, J. Z., Wang, J. J., Zhang, Z. G., & Guo, S. P. (2011). Forecasting stock indices with back propagation neural network. *Expert Systems with Applications*, 38(11), 14346–14355.

- [10] Guresen, E., Kayakutlu, G., & Daim, T. U. (2011). Using artificial neural network models in stock market index prediction. *Expert Systems with Applications*, 38(8), 10389-10397.
- [11] Stanley, K. O., & Miikkulainen, R. (2002). Evolving Neural Networks through Augmenting Topologies. *Evolutionary Computation*, 10(2), 99–127.
<https://doi.org/10.1162/106365602320169811>

Appendix

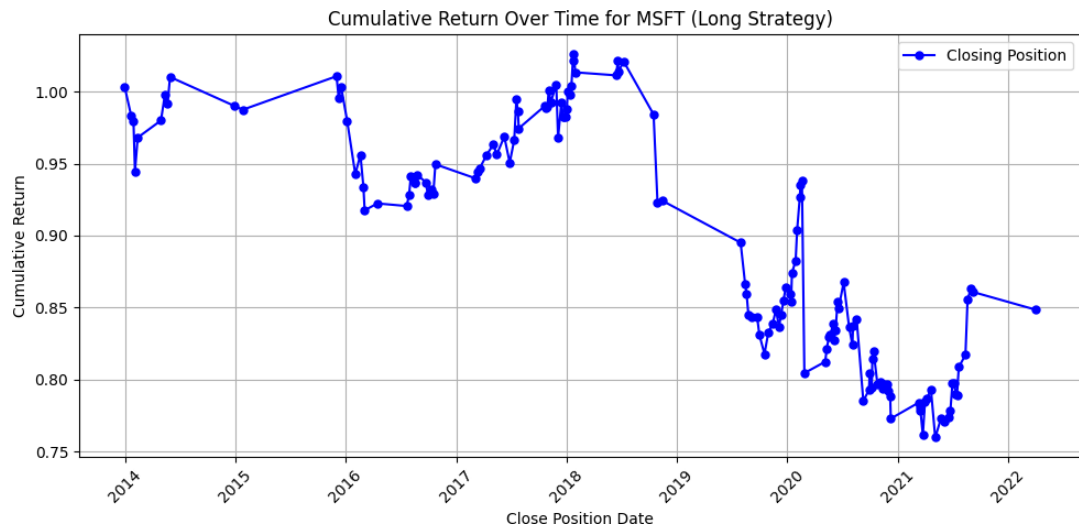
Hyperparameter

Population size of genomes	5000
Training windows size	120
Testing windows size	40
EMA period	3
RSI period	3
Starting balance	100000

The graph of cumulative return over time of Time-weighted adaptive NEAT:

Long Strategy







Short Strategy

Given that the majority of stocks within the Dow Jones Industrial 30 have experienced significant uptrends over the past decade, the frequency of short trades is considerably lower. Consequently, some graphical representations of these trades may appear atypical.

